

Lecture 23 – Attention Mechanisms

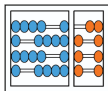
[D2L 10.7, 11.1–11.4]

MC886 – Machine Learning

Marcelo S. Reis [†]

[†] Departamento de Sistemas de Informação
Instituto de Computação, Universidade Estadual de Campinas

Campinas, June 22, 2026



**Instituto de
Computação**

UNIVERSIDADE ESTADUAL DE CAMPINAS



UNICAMP

Summary

Seq2seq learning for machine translation

Queries, keys and values

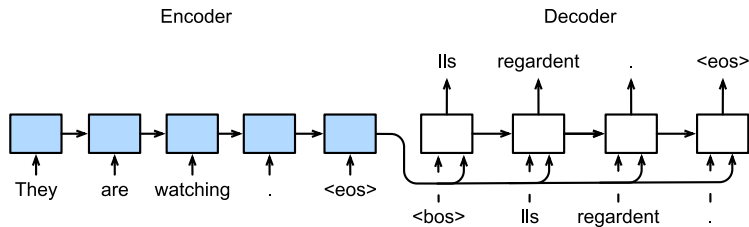
Attention scoring functions

The Bahdanau attention mechanism

Sequence-to-sequence learning for machine translation

- ❖ In seq-2-seq learning, inputs and outputs consist of variable-length unaligned sequences
- ❖ We often resort to the encoder-decoder architecture
- ❖ We can implement the encoder and the decoder using recurrent neural networks (RNNs)

Seq-2-seq learning with RNNs as encoder and decoder



<eos>: end of sequence token

<bos>: beginning of sequence token

Encoder

- ❖ Suppose the input sequence $x_1, \dots, x_T$, such that x_t is the t -th token. The t -th hidden state $\mathbf{h}_t$ of the RNN can be computed as:

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}), \quad (10.7.1)$$

where $\mathbf{x}_t$ is t -th token and $\mathbf{h}_{t-1}$ is the previous hidden state

- ❖ In general, the encoder transforms hidden states of all time steps into a context variable $\mathbf{c}$ which will be passed to the decoder:

$$\mathbf{c} = q(\mathbf{h}_1, \dots, \mathbf{h}_T) \quad (10.7.2)$$

- ❖ In the figure example, $q(\mathbf{h}_1, \dots, \mathbf{h}_T) = \mathbf{h}_T$

Decoder

- ❖ Given the output sequence $y_1, \dots, y_{T'}$, the decoder computes:

$$P(y_{t'+1}|y_1, \dots, y_{t'}, \mathbf{c}),$$

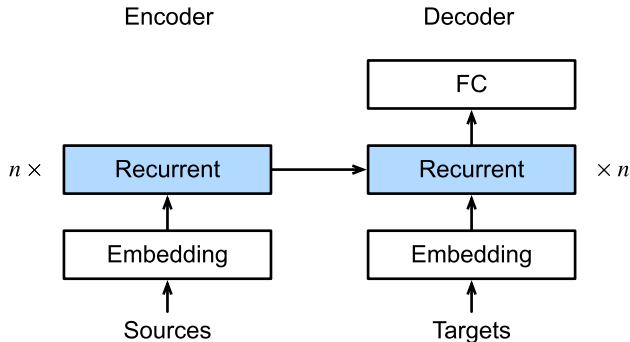
i.e., the probability that token $y_{t'+1}$ occurs at step $t' + 1$ given $y_1, \dots, y_{t'}$ and the context variable $\mathbf{c}$

- ❖ To predict token at t' , we use the previous step predicted token $y_{t'-1}$, the context variable $\mathbf{c}$, and the previous hidden state $\mathbf{s}_{t'-1}$ and compute the current hidden state:

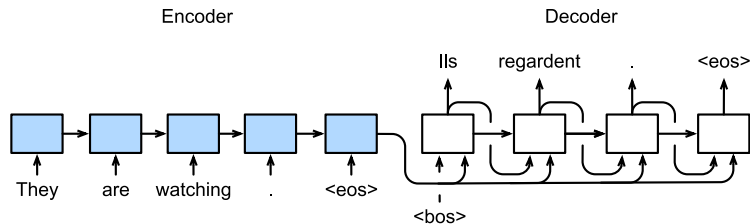
$$\mathbf{s}_{t'} = g(y_{t'-1}, \mathbf{c}, \mathbf{s}_{t'-1}) \quad (10.7.3)$$

- With $\mathbf{s}_{t'}$, we can use an output layer O and the softmax function to compute the distribution $p(y_{t'+1}|y_1, \dots, y_{t'}, \mathbf{c})$ and define $y_{t'+1}$ (softmax's output with largest value)

Stacked layers of a seq-2-seq RNN encoder-decoder



Prediction with the trained RNN encoder-decoder



<eos>: end of sequence token

<bos>: beginning of sequence token

Limitations of this RNN encoder-decoder

- ❖ A variable-length input sequence is transformed into a fixed-shape hidden state vector (context variable $\mathbf{c}$)
- ❖ Output sequence is generated one token at a time, using the hidden state vector and the predicted preceding word (or `<bos>` in the case of predicting the first word)
- ❖ Information used for prediction is compressed into a single fixed-length representation (*information loss*)
- ❖ How to keep track of everything that has already been viewed by the model without compressing the entire input?

Tracking information through the database formalism

- ❖ A database (e.g., the dictionary in Python) is a way to track information on a data sequence
- ❖ A simple database is composed of *keys* (k) and *values* (v), which are organized into a *tuple* (k, v)
- ❖ For example, a database $\mathcal{D}$ with the tuples:

$\{(Zhang, Aston), (Lipton, Zachary), (Li, Mu),$
 $(Smola, Alex), (Hu, Rachel), (Werness, Brent)\}$

- ❖ We can design a *query* (q) to track tuples in $\mathcal{D}$
 - For instance, the exact query for “Li” retrieves “Mu”, while an approximate query for “Li” also returns “Zachary”

More on the queries

- ❖ We can design queries that operate on (k, v) that are valid regardless of the database size
- ❖ The same query can receive different answers, according to the contents of the database
- ❖ The “code” being executed for database queries can be quite simple (e.g., exact match, approximate match, top- k)
- ❖ **No need of data compression!**

Attention, please!

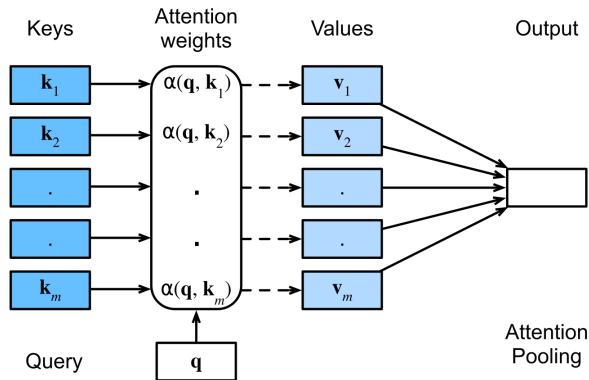
- ❖ Let $\mathcal{D} = \{(\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_m, \mathbf{v}_m)\}$ be a database of m tuples, where $\mathbf{k}_i \in \mathbb{R}^k$ and $\mathbf{v}_i \in \mathbb{R}^v$
- ❖ Moreover, let $\mathbf{q} \in \mathbb{R}^q$ be a *query*
- ❖ The *attention* over $\mathcal{D}$ is defined as:

$$Attention(\mathbf{q}, \mathcal{D}) = \sum_{i=1}^m \alpha(\mathbf{q}, \mathbf{k}_i) \mathbf{v}_i, \quad (11.1.1)$$

where $Attention(\mathbf{q}, \mathcal{D}) \in \mathbb{R}^v$ and $\alpha(\mathbf{q}, \mathbf{k}_i) \in \mathbb{R}$ is a scalar attention weight

- ❖ This operation, also known as *attention pooling*, pays more attention for terms whose α is large (significant)

The attention mechanism



$$Attention(\mathbf{q}, \mathcal{D}) = \sum_{i=1}^m \alpha(\mathbf{q}, \mathbf{k}_i) \mathbf{v}_i \quad (11.1.1)$$

Examples of weights $\alpha(\mathbf{q}, \mathbf{k}_i)$

- ❖ All weights $\alpha(\mathbf{q}, \mathbf{k}_i)$ are non-negative
- ❖ The weights $\alpha(\mathbf{q}, \mathbf{k}_i)$ form a convex combination, i.e., $\sum_{i=1}^m \alpha(\mathbf{q}, \mathbf{k}_i) = 1$ and $\alpha(\mathbf{q}, \mathbf{k}_i) \geq 0$ for all i
- ❖ Exactly one $\alpha(\mathbf{q}, \mathbf{k}_i)$ is 1, all the others are zero (akin to database query)
- ❖ $\alpha(\mathbf{q}, \mathbf{k}_i) = \frac{1}{m}$ for all i (average pooling)

Implementing the weights to form a convex combination

- ❖ We can ensure the weights sum up to 1 via:

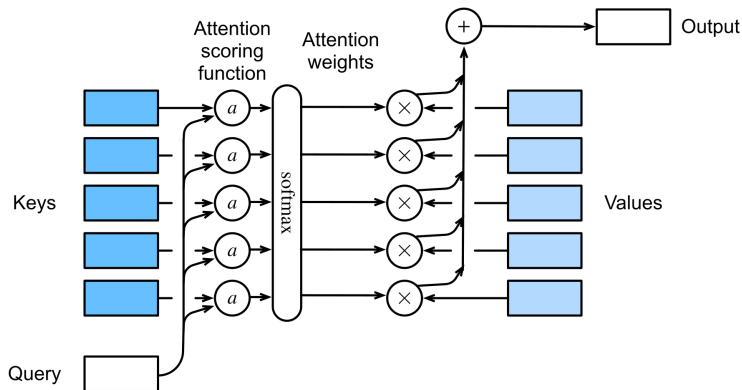
$$\alpha(\mathbf{q}, \mathbf{k}_i) = \frac{\alpha(\mathbf{q}, \mathbf{k}_i)}{\sum_j \alpha(\mathbf{q}, \mathbf{k}_j)} \quad (11.1.2)$$

- ❖ However, using exponentiation to get non-negative values allows us to choose *any* function $a(\mathbf{q}, \mathbf{k}_i)$ and then apply the softmax:

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_j \exp(a(\mathbf{q}, \mathbf{k}_j))} \quad (11.1.3)$$

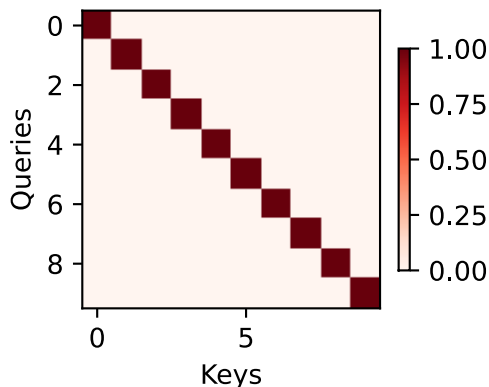
- ❖ Note that this function is differentiable!

Attention function using softmax



$$\text{Attention}(\mathbf{q}, \mathcal{D}) = \sum_{i=1}^m \alpha(\mathbf{q}, \mathbf{k}_i) \mathbf{v}_i = \sum_{i=1}^m \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_j \exp(a(\mathbf{q}, \mathbf{k}_j))} \mathbf{v}_i$$

Visualizing softmax-based attention weights



We interpret large weights as a way for the model to select components of relevance

Additive attention weight

- ❖ We can define the function $a(\mathbf{q}, \mathbf{k})$ as:

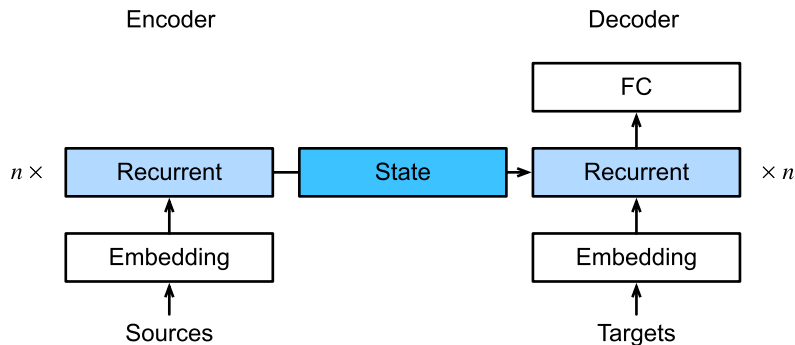
$$a(\mathbf{q}, \mathbf{k}) = \mathbf{w}_v^T \tanh(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k}) \in \mathbb{R}, \quad (11.3.7)$$

where $\mathbf{W}_q \in \mathbb{R}^{h \times q}$, $\mathbf{W}_k \in \mathbb{R}^{h \times k}$ and $\mathbf{w}_v \in \mathbb{R}^h$ are learnable parameters

- ❖ Therefore, we compute an *additive attention weight* α as:

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \text{softmax}(a(\mathbf{q}, \mathbf{k}_i)) = \frac{\exp(\mathbf{w}_v^T \tanh(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k}))}{\sum_j \exp(\mathbf{w}_v^T \tanh(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k}))}$$

Recalling the sequence to sequence model...



Only the state (context variable $\mathbf{c}$) is shared between encoder/decoder

...and limitations of fixed-length state representation

- ❖ That fixed-length context variable $\mathbf{c}$ is the only information from the input sequence used by the decoder
- ❖ While it can work for short sequences, it clearly would fail for long ones
 - There won't be enough "space" in $\mathbf{c}$ for storing all relevant info from the input
- ❖ The decoder will fail to translate long and complex sentences

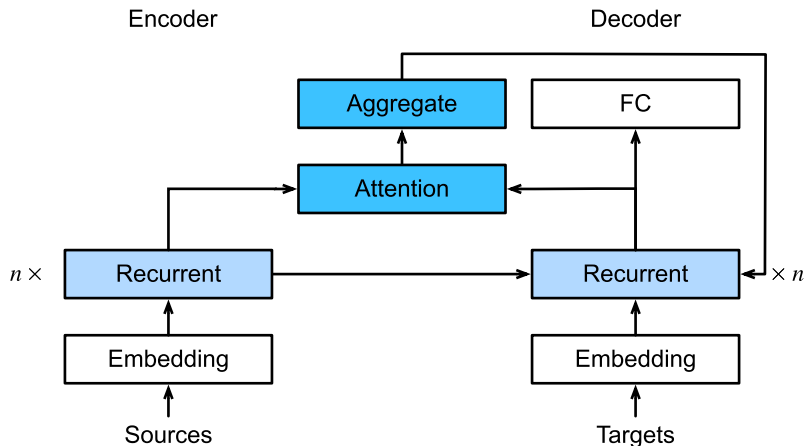
The Bahdanau Attention Mechanism

- ❖ Instead of a single context variable $\mathbf{c}$ to summarize the whole input sentence, the idea is to dynamically update it using the attention pooling:

$$\mathbf{c}_{t'} = \sum_{t=1}^T \alpha(\mathbf{s}_{t'-1}, \mathbf{h}_t) \mathbf{h}_t, \quad (11.4.1)$$

where $\mathbf{s}_{t'-1}$ is the query, and $\mathbf{h}_t$ is both the key and the value

The previous RNN encoder-decoder with the Bahdanau Attention Mechanism



Some remarks

- ❖ Attention mechanisms dynamically extract context from the input, “paying attention” for the most relevant part of the input for the t -th prediction
- ❖ RNN encoder-decoder with the Bahdanau attention mechanism selectively aggregates different parts of the input sequence using the additive attention
- ❖ However, there is a simpler, computationally more efficient attention mechanism: the *scale dot product attention*

Scaled dot product attention weight

- ❖ Let us assume that all $\mathbf{q}, \mathbf{k}_i \in \mathbb{R}^p$ are iid drawn from random variables with zero mean and unit variance
- ❖ In this case, the dot product $\mathbf{q}^T \mathbf{k}_i$ would have zero mean and a variance of p .
- ❖ To keep in track the variance, we compute a *scaled* dot product:

$$a(\mathbf{q}, \mathbf{k}_i) = \mathbf{q}^T \mathbf{k}_i / \sqrt{p}, \quad (11.3.2)$$

- ❖ Therefore, an attention weight α is computed as:

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \text{softmax}(a(\mathbf{q}, \mathbf{k}_i)) = \frac{\exp(\mathbf{q}^T \mathbf{k}_i / \sqrt{p})}{\sum_j \exp(\mathbf{q}^T \mathbf{k}_j / \sqrt{p})} \quad (11.3.3)$$

Computing a batch of scaled dot product attentions

- ❖ In practice, we compute at once n queries and m key-value pairs, with p -sized queries and keys and v -sized values
- ❖ Therefore, we can rewrite the scale dot product attention as:

$$\text{softmax} \left(\frac{QK^T}{\sqrt{p}} \right) \mathbf{V}, \quad (11.3.6)$$

where:

- $\mathbf{Q} \in \mathbb{R}^{n \times p}$ is the set of queries
- $\mathbf{K} \in \mathbb{R}^{m \times p}$ is the set of keys
- $\mathbf{V} \in \mathbb{R}^{m \times v}$ is the set of values